REVIVEOS — Master Build Specification

Track 03 — AI Revenue Recovery

One-line product definition

ReviveOS is an AI revenue-recovery orchestration system that diagnoses failed revenue events, predicts recoverability, selects and schedules the next-best intervention, executes it within strict safety policies, verifies the outcome, and measures incremental revenue recovered against a baseline.

1. The actual problem

A failed payment doesn't tell you what should happen next.

Today, a simplistic recovery workflow looks like:

Payment failed
↓
Retry tomorrow
↓
Retry again
↓
Send reminder
↓
Give up

That's inefficient.

Different failures require different responses:

|

| **Failure** | **Likely response** |

| Temporary bank/network failure | Delayed retry |

| Insufficient funds | Retry at better time |

| Expired card | Payment-method update |

| Authentication failure | Customer action |

| Repeated failures | Escalation |

| Customer opted out | Stop |

| Payment eventually succeeds | Stop recovery |

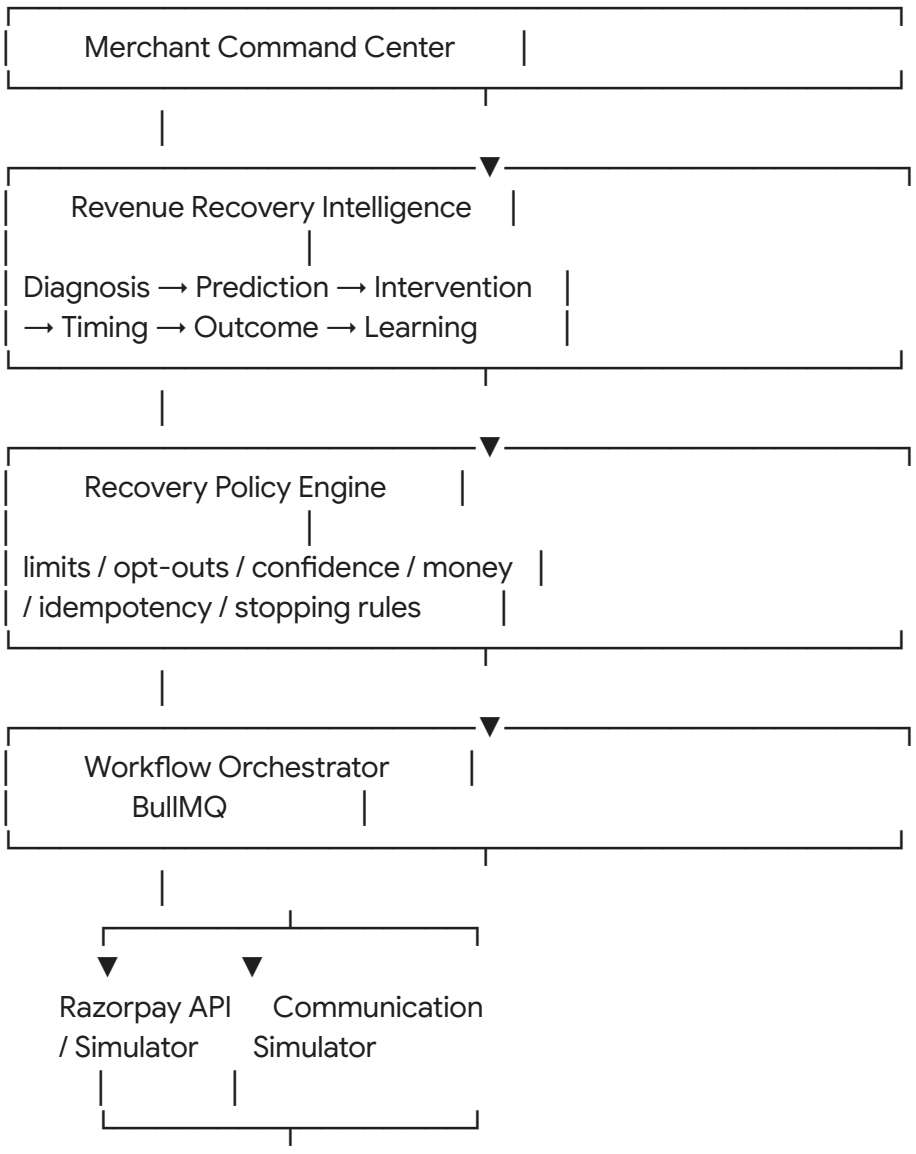
| Unknown/ambiguous | Human review |

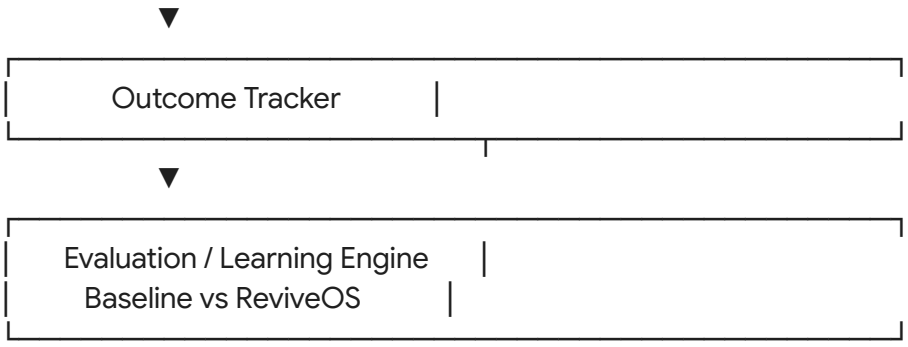
Your problem is therefore:

How can a payment platform intelligently determine the best recovery action, timing and stopping condition for every revenue-loss event, while safely executing it and proving how much incremental revenue it recovered?

2. Product architecture

Build these 7 layers:





3. Feature Pillar 1 — Adaptive Recovery Orchestrator

This is your main AI feature.

It answers four questions:

1. What happened?
2. Can we recover it?
3. What should we do?
4. When should we do it?

The workflow:

PAYMENT FAILED
↓
Normalize event
↓
Determine failure class
↓
Build customer context
↓
Predict recovery probability
↓
Generate candidate interventions
↓
Rank interventions
↓
Select best intervention
↓
Determine optimal timing
↓
Policy validation
↓

Schedule
↓
Execute
↓
Verify

4. Failure Intelligence

Do not let the LLM be your raw payment-failure classifier. Use three layers:

Layer A — Deterministic classifier

Map known gateway/payment errors into categories.

Example:

- NETWORK_ERROR
- BANK_UNAVAILABLE
- TIMEOUT
- INSUFFICIENT_FUNDS
- EXPIRED_CARD
- AUTHENTICATION_FAILED
- LIMIT_EXCEEDED
- MANDATE_FAILED
- UNKNOWN

Layer B — Historical intelligence

Calculate:

- customer_success_rate
- customer_failure_rate
- previous_recovery_rate
- average_payment_amount
- attempt_count
- days_since_last_success
- preferred_payment_method
- historical_recovery_time

Layer C — LLM reasoning

Use DeepSeek-R1 locally through Ollama if you want that architecture.

Input:

{

```
 "failure": "...",
 "customer_context": "...",
 "payment_history": "...",
 "merchant_policy": "..."
}
```

Output strict JSON, for example:

```
{
 "diagnosis": "temporary_bank_failure",
 "recoverability": 0.87,
 "recommended_action": "DELAYED_RETRY",
 "recommended_delay_hours": 2,
 "reason": "Transient infrastructure failure with high historical recovery probability",
 "confidence": 0.91
}
```

Then your deterministic policy engine validates this.

LLM proposes. Policy engine disposes.

5. Recovery probability model

You need an actual number: $P(\text{recovery})$

Start simple. You can use:

- Logistic regression
- XGBoost
- LightGBM

Features:

- failure_code
- amount
- payment_method
- attempt_number
- customer_success_rate
- customer_failure_rate
- days_since_last_payment
- subscription_age
- historical_recovery_rate

- time_of_day
- day_of_month
- merchant_category

Output:

$$\text{Recovery probability} = 78\%$$

Then compare your model against a baseline.

6. Next-Best-Action Engine

Your candidate actions:

- **A. Immediate retry:** For highly transient issues.
- **B. Delayed retry:** E.g., $+2 \text{ hours}$, $+6 \text{ hours}$, $+24 \text{ hours}$.
- **C. Payment-method update:** For expired card, blocked card, or invalid payment method.
- **D. Payment link:** Give the customer a fresh way to complete payment.
- **E. Customer notification:** Initially implement Email and simulated WhatsApp. Don't build five communication integrations.
- **F. Escalation:** Send to human operations.
- **G. No action:** This is important. Sometimes the best intervention is doing nothing.

7. Dynamic Timing Engine

Don't only select: *"Retry."* Select: *"Retry at X."*

Your synthetic data should contain behavioral patterns.

Example:

- Customer historically pays: 1st–3rd of month
- Payment failed: 29th
- System: Schedule retry for 1st 10:00 AM

Your benchmark can then show:

- Static retry: 22% recovered
- Dynamic timing: 37% recovered
- Lift: $+15$ percentage points

(These numbers are examples only; your actual system must generate and measure its own results.)

8. BullMQ Workflow Orchestrator

Use BullMQ for:

- Delayed retries
- Scheduled interventions
- Communication jobs
- Verification jobs
- Recovery workflows
- Retry/backoff
- Dead-letter jobs

recovery:analyze
↓
recovery:plan
↓
recovery:policy
↓
recovery:schedule
↓
recovery:execute
↓
recovery:verify
↓
recovery:complete

Use job IDs based on a stable workflow/payment identity to support idempotency.

9. CRITICAL — Recovery State Reconciliation

This should absolutely be in your final build.

A payment can appear failed and subsequently succeed, so your system must verify the current state before contacting/retrying the customer. The supplied plan specifically identifies this payment.failed → wait/verify → recovered/continue behavior as a production-grade concern.

payment.failed
↓
WAIT / VERIFY
↓
Current state?
┌──────┐
SUCCESS FAILED
| |

STOP RECOVERY

This prevents embarrassing behavior such as: *Customer successfully paid → ReviveOS immediately sends "Your payment failed" WhatsApp.*

That would be a great demo failure to prevent.

10. Feature Pillar 2 — Recovery Policy & Safety Engine

This is your trust layer. Every AI decision passes through:

AI recommendation
↓
Policy Engine
↓
ALLOW / MODIFY / BLOCK / ESCALATE

11. Stopping rules

Implement:

- **Maximum payment attempts:** $\text{MAX\_RETRIES} = 3$
- **Maximum communications:** $\text{MAX\_CONTACTS} = 2$, $\text{MAX\_CONTACTS\_WINDOW} = 72\text{h}$
- **Maximum workflow lifetime:** E.g., $\text{MAX\_RECOVERY\_WINDOW} = 7\ \text{days}$
- **Customer opt-out:** STOP / UNSUBSCRIBE / DO NOT CONTACT $\rightarrow$ immediately block communication.
- **Confidence threshold:** $\text{confidence} < 0.70 \rightarrow$ human review.
- **Monetary threshold:** E.g., $\text{amount} > ₹50,000 \rightarrow$ human approval.

(The exact values are your policy choices, not Razorpay requirements; document them clearly.)

12. Idempotency protection

This is a major production-readiness feature. Imagine:

- Worker A $\rightarrow$ retry payment

- Worker B $\rightarrow$ retry payment ...at the same time.

You must prevent double execution.

Use: **Redis distributed lock + idempotency key + PostgreSQL state**

Example key: recovery:{payment_id}:{action}:{attempt}

If the same job arrives twice: Already executed $\rightarrow$ do not execute again.

13. Redis responsibilities

Use Redis for:

x

- BullMQ

x

- Distributed locks

x

- Rate limiting

x

- Idempotency

x

- Short-lived workflow state

x

- Deduplication

Don't use Redis as your permanent source of truth.

14. PostgreSQL responsibilities

PostgreSQL stores:

- customers
- merchants
- payments
- subscriptions
- failure_events
- recovery_workflows
- recovery_actions
- recovery_outcomes
- policies
- customer_preferences
- audit_events
- experiments

15. Tamper-evident audit ledger

Don't simply create audit_logs. Make it interesting:

audit_events:

- event_id
- workflow_id
- timestamp
- actor
- action
- payload_hash
- previous_event_hash
- event_hash

Each event references the previous event:

Event 1
↓ hash
Event 2
↓ hash
Event 3
↓ hash
Event 4

Now you can demonstrate: *"Every recovery decision can be reconstructed and the event chain is tamper-evident."*

16. Human Operations Queue

When the system can't safely decide:

AI
↓ (confidence = 0.52)
POLICY
↓
HUMAN REVIEW

Dashboard:

> **HIGH PRIORITY**
>
> Payment: pay_82912
>
> Amount: ₹87,000
>
> Reason: Repeated failures + ambiguous gateway response
>
> AI confidence: 52%
>
> Recommended: Manual review

Human can:

- [Approve Retry]
- [Send Payment Link]
- [Stop Recovery]

This gives you human-in-the-loop control.

17. Feature Pillar 3 — Closed-Loop Recovery Intelligence

This is arguably the most important judging feature. You don't just claim: *"Our AI recovers money."* You prove it. Build an evaluation engine.

18. Synthetic dataset

Start with:

- **MVP:** 500 events
- **Scale to:** 5,000–10,000 events

(You only need 50+ to satisfy the basic demonstration, but hundreds/thousands make your evaluation credible.)

Each event:

```
{
  "payment_id": "pay_10291",
  "customer_id": "cust_829",
  "amount": 4999,
  "currency": "INR",
  "payment_method": "card",
  "failure_code": "INSUFFICIENT_FUNDS",
  "attempt_number": 1,
  "subscription_id": "sub_991",
  "timestamp": "...",
  "customer_history": {
    "successful_payments": 8,
    "failed_payments": 1
  }
}
```

The supplied plan uses this kind of synthetic event structure for the evaluation dataset.

19. Inject realistic failure patterns

Your generator should create:

- 20% transient failures
- 20% insufficient funds
- 15% expired/payment-method issues
- 10% authentication failures
- 10% mandate failures
- 10% network failures
- 10% customer-action-required
- 5% ambiguous

(These are simulation assumptions, so clearly document them.)

Then generate known ground truth:

- Would recover naturally?
- Would recover after retry?

- Would recover after customer action?
- Would never recover?

This allows objective evaluation.

20. Baseline strategy

You absolutely need this. Implement a simple benchmark:

Baseline:

- T+1 retry
- T+2 retry
- T+3 retry
- then stop

The supplied plan specifically recommends comparing this static strategy against your dynamic strategy so you can measure incremental revenue recovered, rather than merely reporting total recovery.

21. ReviveOS strategy

Your strategy:

Failure diagnosis
+
Recovery probability
+
Customer context
+
Timing
+
Intervention selection
+
Safety policy

Then run the same dataset through both.

22. Your core metrics

Dashboard:

- **Revenue at Risk:** ₹10.0L
- **Recoverable Revenue:** ₹X
- **Revenue Recovered:** ₹X

- **Recovery Rate:** $\frac{\text{Recovered}}{\text{Recoverable}}$
- **Incremental Recovery:** $\text{ReviveOS recovery} - \text{Baseline recovery}$
- **Recovery Lift:** $\frac{\text{ReviveOS} - \text{Baseline}}{\text{Baseline}}$
- **Unnecessary Interventions:** Actions that didn't need to happen
- **Failed Interventions:** Actions attempted but unsuccessful
- **Escalation Rate:** $\frac{\text{Human-review cases}}{\text{Total cases}}$
- **Policy Violations Target:** 0

23. Outcome attribution

Every intervention needs:

- action_id
- payment_id
- action_type
- scheduled_at
- executed_at
- result
- amount
- recovered_amount
- time_to_recovery

Then you can answer: *"This ₹4,999 recovery came from this specific intervention."*

That's much stronger than simply seeing the payment eventually succeed.

24. Recovery Learning Engine

Once outcomes arrive:

Action
↓
Outcome
↓
Update statistics
↓
Improve future recommendation

For example:

- Failure type: INSUFFICIENT_FUNDS
    - Retry +24h: 34% recovery

- Retry +48h: 21%
- Payment link: 17%
- WhatsApp + retry: 38%

Then your next decision can incorporate those empirical results. You don't need sophisticated reinforcement learning. A contextual bandit or even statistically tracked action performance is enough for the MVP.

25. Batch command center

This is your video centerpiece.

Top:

> **REVIVEOS**
>
> Revenue Recovery Command Center

Cards:

- ₹18.42L Revenue at Risk
- ₹12.73L Recoverable
- ₹7.84L Recovered
- 61.6% Recovery Rate

Funnel:

REVENUE AT RISK
₹18.42L
↓
DIAGNOSED
₹18.42L
↓
INTERVENTION ACTIVE
₹11.31L
↓
RECOVERED
₹7.84L

Then: **BASELINE VS REVIVEOS**

26. Individual recovery timeline

Click any transaction. Show:

PAY_10291 (₹4,999)

10:31:02 Payment failed

10:31:03 Failure classified: Insufficient funds

10:31:04 Recovery probability: 78%

10:31:04 Next best action: Delayed retry

10:31:05 Policy: APPROVED

10:31:05 Scheduled: +24 hours

NEXT DAY

10:00:01 Retry executed

10:00:03 Payment captured

RECOVERED ₹4,999

This is excellent for the pitch.

27. Tiny merchant simulator

Do not build a full ecommerce application. Build a tiny simulator:

```
┌──────────────────────────────────────┐
│ REVIVEOS PAYMENT SIMULATOR │
├──────────────────────────────────────┤
│ Customer: Rahul Sharma     │
│ Subscription: ₹4,999/mo    │
│ Payment method: Card       │
│                            │
│ [ Simulate Failure ]       │
│ [ Fast Forward ]           │
│ [ Simulate Success ]       │
└──────────────────────────────────────┘
```

The supplied plan explicitly recommends a small simulator for the pitch rather than making the checkout itself the product.

28. Real Razorpay integration vs simulator

Use both, but don't confuse their roles.

Razorpay Test Mode

Use it where practical for:

- API integration
- Webhook handling
- Payment lifecycle
- Realistic request/response flow

Your simulator

Use for:

- Deterministic failure scenarios
- Fast-forwarding time
- Generating controlled failures
- Reproducing edge cases
- Pitch video

Synthetic dataset

Use for:

- 500–10,000 event evaluation
- Baseline comparison
- Statistical metrics

That gives you: real API + controlled demo + measurable benchmark.

29. Tech stack

Recommended setup:

- **Frontend:** Next.js + TypeScript
- **Backend:** Node.js + TypeScript (ideal for BullMQ orchestration)
- **Queue:** BullMQ
- **Queue backend:** Redis
- **Database:** Supabase PostgreSQL
- **AI:** DeepSeek-R1 via Ollama
- **ML:** Python microservice or initially a simple model inside the backend
- **API:** Razorpay Test Mode
- **Charts:** Recharts
- **Deployment:** Whatever you can reliably deploy quickly (don't spend two days fighting Kubernetes).

30. Suggested repository structure

```
reviveos/
│
├── apps/
```

```
│   ├── dashboard/
│   │   ├── app/
│   │   ├── components/
│   │   ├── charts/
│   │   └── simulator/
│   │
│   └── api/
│       ├── routes/
│       ├── controllers/
│       └── middleware/
│
├── services/
│   ├── recovery-engine/
│   ├── policy-engine/
│   ├── outcome-engine/
│   ├── evaluation-engine/
│   └── ml-service/
│
├── workers/
│   ├── recovery.worker.ts
│   ├── retry.worker.ts
│   ├── communication.worker.ts
│   └── verification.worker.ts
│
├── packages/
│   ├── types/
│   ├── policy/
│   ├── schemas/
│   └── utils/
│
├── simulator/
│   ├── scenarios/
│   ├── generator/
│   └── fixtures/
│
├── ml/
│   ├── training/
│   ├── inference/
│   └── evaluation/
│
├── database/
│   ├── migrations/
│   └── seed/
```

```
│
├── docs/
│   ├── architecture.md
│   ├── decisions.md
│   ├── failure-modes.md
│   └── evaluation.md
│
├── docker-compose.yml
├── README.md
└── package.json
```

31. Database schema

Minimum structure:

```
merchants
———————————
id
name
policies

customers
———————————
id
merchant_id
preferences
payment_profile

subscriptions
———————————————
id
customer_id
amount
status
next_billing_at

payments
————————
id
customer_id
subscription_id
amount
```

status
method
failure_code
created_at

recovery_workflows
_____

id
payment_id
status
recovery_probability
selected_action
scheduled_at

recovery_actions
_____

id
workflow_id
action_type
status
attempt
executed_at
result

recovery_outcomes
_____

id
action_id
payment_id
recovered
recovered_amount
time_to_recovery

policies
_____

id
max_retries
max_contacts
max_recovery_window
confidence_threshold
amount_threshold

customer_preferences

customer_id
communication_opt_out

audit_events
id
workflow_id
timestamp
actor
action
payload_hash
previous_event_hash
event_hash
metadata

experiments
id
name
strategy
batch_id

experiment_results
experiment_id
payment_id
strategy
recovered
recovered_amount

32. API endpoints

- POST /webhooks/razorpay
- POST /recovery/analyze/:paymentId
- POST /recovery/plan/:workflowId
- POST /recovery/execute/:actionId
- POST /recovery/stop/:workflowId
- POST /customer/:id/opt-out
- GET /recovery/:id
- GET /recovery/:id/audit

- GET /analytics/overview
- GET /analytics/baseline-vs-ai
- POST /simulator/failure
- POST /simulator/fast-forward
- POST /simulator/success

33. Recovery state machine

Make this explicit in code:

```
FAILED
 ↓
PENDING_VERIFICATION
 ↓
ANALYZING
 ↓
PLANNED
 ↓
POLICY_CHECK
 ↓
SCHEDULED
 ↓
EXECUTING
 ↓
VERIFYING
  ├──→ RECOVERED
  ├──→ NEXT_ATTEMPT
  ├──→ ESCALATED
  └──→ HALTED
```

Never allow arbitrary transitions.

34. Failure scenarios you MUST test

Create automated tests for:

1. **Scenario 1:** Payment fails $\rightarrow$ retry $\rightarrow$ success.
2. **Scenario 2:** Payment fails $\rightarrow$ eventually succeeds before intervention (system must not contact customer unnecessarily).
3. **Scenario 3:** Payment fails three times $\rightarrow$ Stop.
4. **Scenario 4:** Customer says STOP $\rightarrow$ All communications blocked.

5. **Scenario 5:** Two workers process the same payment $\rightarrow$ One action.
6. **Scenario 6:** API timeout $\rightarrow$ Retry safely without duplicate financial action.
7. **Scenario 7:** LLM returns invalid JSON $\rightarrow$ Recovery fails safely.
8. **Scenario 8:** LLM recommends an action violating policy $\rightarrow$ Policy engine blocks it.
9. **Scenario 9:** Low-confidence diagnosis $\rightarrow$ Human escalation.
10. **Scenario 10:** High-value payment $\rightarrow$ Human approval.

These are the scenarios that make your "production-ready" claim believable.

35. What you should deliberately break for the video

Pick 2–3:

- **Break #1 — Duplicate worker:** Two jobs $\rightarrow$ same payment $\rightarrow$ Redis lock $\rightarrow$ one execution.
- **Break #2 — LLM unsafe recommendation:** LLM says *"Retry indefinitely."* Policy: $\text{MAX\_RETRIES} = 3 \rightarrow$ **BLOCKED**.
- **Break #3 — Payment recovered unexpectedly:** payment.failed $\rightarrow$ before recovery action $\rightarrow$ payment.captured $\rightarrow$ workflow automatically terminated.

(That third one is especially strong because it demonstrates why state reconciliation exists.)

36. What NOT to build

Do not waste time on:

- Voice calls
- Five messaging integrations
- A huge merchant portal
- Mobile app
- Kubernetes
- 10 different AI agents
- Custom foundation model
- Blockchain
- Complicated RAG
- Production payment processing
- Beautiful animations everywhere

Your complexity should be inside the decision and safety loop, not the UI.

37. Your final feature hierarchy

🔴 MUST HAVE (Non-negotiable)

- Payment failure ingestion
- Failure diagnosis
- Recovery probability
- Next-best intervention
- Dynamic timing
- BullMQ orchestration
- Policy engine
- Stopping rules
- Idempotency
- State reconciliation
- Outcome tracking
- Synthetic batch
- Baseline strategy
- ReviveOS strategy
- Incremental revenue metric
- Audit trail
- Dashboard
- Human escalation

🟠 HIGH-IMPACT (Add after MUST HAVE works)

- Customer behavioral timing
- Payment-method switching
- Payment-link intervention
- Learning from outcomes
- A/B experiment engine
- Recovery strategy comparison
- Tamper-evident audit hashes
- Failure injection framework
- Merchant policy customization

🟢 OPTIONAL (Only if everything else is finished)

- Contextual bandit
- WhatsApp simulation
- Advanced ML model
- Multiple merchant profiles
- Strategy auto-optimization
- More sophisticated forecasting

38. The one thing I want you to obsess over

Your final GitHub README should be able to answer this in 30 seconds:

> **"Did this actually recover more money than a normal retry strategy?"**

And your answer should be a table:

| **Metric** | **Baseline** | **ReviveOS** |

| Revenue at risk | ₹X | ₹X |

| Revenue recovered | ₹X | ₹X |

| Recovery rate | X% | X% |

| Unnecessary interventions | X | X |

| Failed interventions | X | X |

| Avg. recovery time | Xh | Xh |

| Policy violations | X | 0 |

(The actual values must come from your experiment, not invented numbers. That is the heart of the entire project.)

39. Final 5-minute demo structure

- **0:00–0:30 — Problem:** *"A payment failure doesn't tell a merchant what to do next."* Show: ₹18.42L Revenue at Risk.
- **0:30–1:15 — Live failure:** Simulate ₹4,999 subscription INSUFFICIENT_FUNDS. Show: Diagnosis → 78% recovery probability → Delayed retry → 24h timing.
- **1:15–2:00 — Safety:** Show policy check (✓ retry allowed, ✓ customer not opted out, ✓ attempt < 3, ✓ confidence > threshold). Schedule BullMQ job.
- **2:00–2:45 — Recovery:** Fast-forward. Payment succeeds. ₹4,999 RECOVERED. Audit timeline appears.
- **2:45–3:30 — Break it:** Simulate duplicate job / payment already recovered. System stops duplicate action.
- **3:30–4:20 — Batch:** Run 500 / 1,000 synthetic failures. Show: BASELINE vs REVIVEOS (+Z% incremental recovery).
- **4:20–5:00 — Close:** Show Revenue at risk → Diagnosis → Next-best action → Safe execution → Verification → Revenue recovered → Learning. Say: *"ReviveOS doesn't just retry failed payments. It decides how, when and whether to recover them, executes within strict financial controls, and measures the incremental revenue its*

decisions create."

The build order I recommend

Do not start with the dashboard. Build in this order:

- **Day 1:** Database + event schema + synthetic generator
- **Day 2:** Failure classifier + recovery state machine
- **Day 3:** Recovery probability + next-best-action engine
- **Day 4:** BullMQ + delayed jobs + idempotency
- **Day 5:** Policy/safety engine + audit ledger
- **Day 6:** Razorpay test-mode/webhook integration
- **Day 7:** Outcome tracker + state reconciliation
- **Day 8:** Baseline vs ReviveOS evaluation
- **Day 9:** Dashboard
- **Day 10:** Simulator + failure injection
- **Day 11:** Hardening + tests + observability
- **Day 12:** README + architecture + pitch
- **Day 13:** Final polish + recording

The critical dependency is: don't build the UI until the complete failure $\rightarrow$ *decision* $\rightarrow$ *policy* $\rightarrow$ *queue* $\rightarrow$ *action* $\rightarrow$ *verification* $\rightarrow$ *recovered loop works end-to-end. That loop, plus the baseline-vs-ReviveOS measurement, is what turns this from a polished demo into a serious engineering submission.*